

University Information Chatbot

A-to-Z Project Process and Technical Documentation

A complete description of how the PDF-grounded multilingual RAG system is structured, executed, evaluated, and maintained.

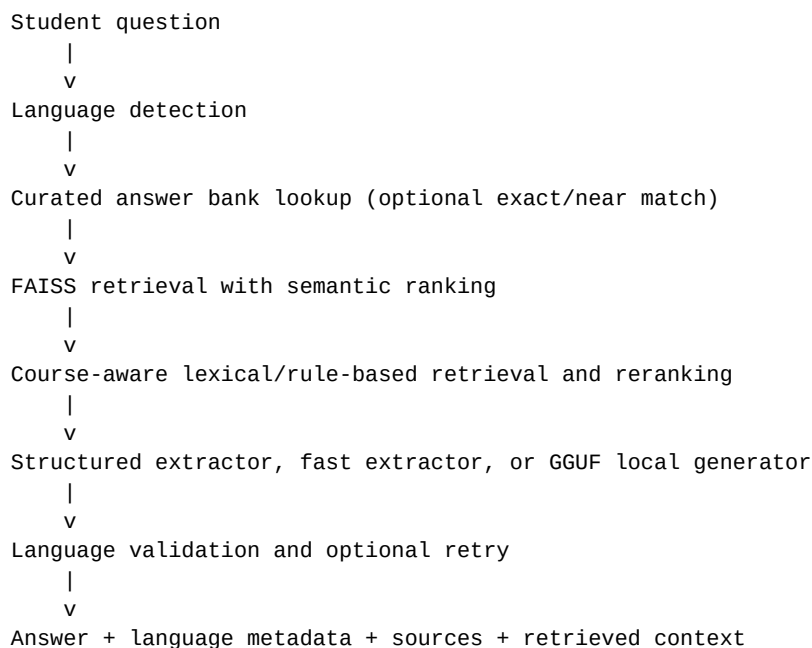
Knowledge base	curricula_BSc-Curriculum-New.pdf
Languages	English, Bangla, Banglish
Default UI mode	Fast extractive answering
Optional mode	Local Qwen2.5 GGUF generation
Vector store	FAISS IndexFlatIP, metadata.pkl, index_manifest.json
Current index	95 chunks

1. Purpose

University Information Chatbot is a thesis prototype that answers questions about university courses and policies using only the official PDF documents stored in `data/documents/`. The system supports English, Bangla, and Banglish questions. It returns a grounded answer, detected language, generation mode, and traceable source/page metadata from the retrieved chunks.

The project implements a retrieval-augmented workflow: questions are routed through language detection, vector retrieval, lexical and rule-based score tuning, answer construction, and language validation. It is designed to answer from university documentation rather than from an unrestricted open-world knowledge source.

2. Current System Architecture



3. Knowledge Sources

The project knowledge base is the collection of official PDFs in `data/documents/`.

Current document stored in the workspace:

- `curricula_BSc-Curriculum-New.pdf` — UAP Department of CSE undergraduate prospectus and B.Sc. curriculum.

The source question workbook is stored as `data/questions/uap_cse_multilingual_rag_dataset.xlsx`. `scripts/import_question_workbook.py` converts it into the canonical evaluation CSV at `data/questions/questions.csv`. The CSV is used for testing and evaluation. It is not included in retrieval context or passed directly to generation, except when the optional curated answer-bank path is enabled.

4. Clean Repository Structure

The project has been reorganized so the active thesis implementation is separated from generated output and legacy artifacts.

Current top-level structure:

app.py	Streamlit application
run_app.py	Streamlit launcher
README.md	Project overview and command reference
requirements.txt	Python dependency list
data/documents/	Active source PDF documents
data/questions/	Canonical dataset and workbook
src/	Runtime RAG implementation
scripts/	Ingestion, indexing, evaluation, and utilities
tests/	Current automated tests
vector_db/	Active FAISS index, metadata, and manifest
docs/	Project process documentation
archive/	Legacy scripts, reports, tests, and old outputs

The current active data files are:

- data/documents/curricula_BSc-Curriculum-New.pdf
- data/questions/uap_cse_multilingual_rag_dataset.xlsx
- data/questions/questions.csv

The active vector store files are:

- vector_db/index.faiss
- vector_db/metadata.pkl
- vector_db/index_manifest.json

Legacy 180-query results, old runtime probes, and old-corpus regression fixtures are kept under archive/ so they remain available for historical inspection without mixing with current 300-variant thesis experiments.

5. Index-Building Process

Run python scripts/build_index.py --force after changing or adding PDF sources.

The current index workflow is:

1. pdf_loader.py reads supported PDF files with pypdf.
2. Page text is extracted and normalized.
3. chunker.py splits the page text into overlapping word windows of 700 words with 120-word overlap; chunks are page-safe.
4. embeddings.py encodes each chunk using BAAI/bge-m3 and stores normalized embeddings.
5. vector_store.py creates a FAISS IndexFlatIP inner-product vector index.
6. The index is saved as vector_db/index.faiss.
7. Metadata records are saved as vector_db/metadata.pkl.

The metadata records include the original text, source, page, chunk_id, word_start, and word_end. The FAISS vector record order corresponds to the metadata record order. The current active index contains 95 chunks.

6. Runtime Query Pipeline

src/pipeline.py is the central orchestration layer for a request.

1. Reject an empty or blank question.

2. Detect the question language using `language_detector.py`.
3. Check the curated answer bank for a high-confidence question similarity match.
4. Load the FAISS index and metadata from `vector_db/`.
5. Select `use_generation=True` for local GGUF generation or `False` for fast extraction.
6. Retrieve semantic candidates and apply lexical, course, objective, and CLO-aware rerank logic.
7. Build an answer using one of the following branches:
 - answer bank match - structured topic extraction route - fast extractive route - local GGUF answer generation route
8. Validate that the answer's detected response language matches the detected question language.
9. If local generation creates the wrong language style, retry once with a stronger language instruction.
10. Return the final answer, response language metadata, source list, page list, scores, and retrieved context.

7. Retrieval Behavior

`src/retriever.py` combines a semantic vector search and a rule-based reranking workflow.

Current retrieval behavior is:

- BAAI/bge-m3 creates the semantic query vector.
- FAISS returns nearest-neighbor chunks using inner-product similarity.
- Question token keywords produce a small positive score contribution.
- Course-code hints such as CSE 101, MTH 101, and ENG (CSE) 101 assign a course-aware boost.
- Same-course chunks are preserved, so later chunks from the same course can support CLO, objective, and metadata-style questions.
- Explicit CLO questions add a strong score boost when the target CLO is present in the chunk.
- Objective/topic-style questions add a strong score boost when course objectives text is found.
- Course mismatches are penalized when the question clearly targets one known course code.

`src/fast_answer.py` also provides lexical retrieval for the fast mode. It scores presence of query tokens, course-code matching, and known answer patterns without loading the embedding model.

8. Answer Modes

8.1 Curated Answer Bank

`src/answer_bank.py` is checked before the semantic index. It uses the curated CSV to return a high-similarity answer when a question is close to a known evaluated example. The answer bank now uses the language-specific ground-truth answer columns from the imported workbook.

This path is not a retrieval answer; it is an offline answer lookup that can provide deterministic formatting and reduced latency for repeated or training-style queries.

8.2 Structured topic extraction

`src/fast_answer.py` contains a pre-generation structured answer layer. If a question asks about under `<subject>` topics, the system can return a structured list of topics from retrieved chunks instead of forcing an LLM answer.

8.3 Fast Extractive Mode

The default fast mode is deterministic and intended for Streamlit UI use. It reads the retrieved document chunks and extracts recognized answer structure from the text.

Supported extraction patterns include:

- Course code and course title metadata
- Course type and credit information
- CLO 1 through CLO 4 numeric pattern answers
- Course objectives answers
- Final Exam and Mid Term percentage answers

If a question does not map to a focused extraction pattern, the fallback is a sentence-level retrieval summary with source labels.

8.4 Local GGUF Generation

When `use_generation=True`, the system loads a local GGUF model via `llama-cpp-python` and uses `LLM_MODEL` from `src/config.py`. The default project path is `models/Qwen2.5-1.5B-Instruct-Q4_K_M.gguf`, but the environment variable `LLM_MODEL_PATH` can override this.

The generated answer is built from the top retrieved chunk context, then sent through language formatting and answer post-processing. It is designed to keep the answer in the detected language and grounded only in supplied context.

9. Language Handling

The language code in the repository supports the three labels:

- `bangla`: Bengali Unicode detection
- `banglish`: Latin-script Bangla markers and patterns
- `english`: Latin text without sufficient Banglish evidence

`language_detector.py` detects the question language. `language_validator.py` checks the answer's response language and returns a failure flag when the output language fails to match. The generator and answer formatter attempt to route answers through the detected-language style.

10. User Interface

`app.py` provides the Streamlit UI.

The UI shows:

- Index readiness
- Backend import or dependency errors
- A single-question input box
- A toggle allowing local GGUF generation
- A progress and elapsed-time display
- Detected language, mode, answer, sources, pages, scores, and expandable retrieved context.

`run_app.py` launches the Streamlit application through the selected Python environment and defaults to port 8511. A custom port can be supplied explicitly, such as:

```
python run_app.py 8501
```

11. Command Reference

Install dependencies:

```
python -m pip install -r requirements.txt
```

Inspect PDF extraction and chunking:

```
python scripts/ingest.py
```

Build or reuse the vector index:

```
python scripts/build_index.py
```

Ask one question with GGUF generation:

```
python scripts/test_query.py "What is CLO 1 of CSE 101?"
```

Ask one question with fast extraction:

```
python scripts/test_query.py --fast "What percentage of the total marks is allocated to the Final Exam?"
```

Run evaluation:

```
python scripts/evaluate.py
```

Run retrieval evaluation:

```
python scripts/evaluate_retrieval.py
```

Run the UI:

```
python run_app.py 8501
```

12. Evaluation Process

src/evaluator.py reads the available dataset rows from data/questions/questions.csv, runs each question through the pipeline, and writes the results to results/evaluation_results.csv.

Each row records the original question, question language variants, generated answer, detected language, response language, generation mode, source and page metadata, retrieval scores, and the answer's language-consistency flag.

The imported dataset contains expected answer pages for the current curriculum PDF. Retrieval metrics can use those page labels, while exact factual-answer quality still requires manual review against the curriculum source.

Current thesis experiments should write new outputs to results/. Historical 180-query outputs from the earlier three-PDF setup are stored separately in archive/legacy_results/.

13. Runtime and Dependency Notes

The repository's dependency stack in requirements.txt currently includes:

- pypdf
- numpy

- sentence-transformers
- faiss-cpu
- transformers
- sentencepiece
- indic-transliteration
- llama-cpp-python
- torch>=2.6.0,<2.13.0
- torchvision>=0.21.0,<0.22.0
- streamlit
- pandas
- openpyxl

The project uses a CPU-friendly local pipeline. The torchvision dependency was added because the Transformers and Streamlit path can inspect optional vision-related modules during startup. Without torchvision, the environment may raise a startup import problem.

A Windows c10.dll or Torch startup problem is a runtime environment issue, not proof that the retrieval or answer pipeline is incorrect.

14. Current Implementation Status and Lessons Learned

The current codebase includes the following concrete implementation improvements:

- Answer-bank matching is integrated before the vector retrieval branch.
- Structured topic extraction is supported in the answer-building path.
- Course-aware retrieval supports course-code hints in the single current curriculum PDF.
- GGUF local generation is handled through llama-cpp-python and the environment variable LLM_MODEL_PATH override.
- Fast extraction continues to support metadata questions such as course code, course title, course type, credits, CLO questions, objective questions, prerequisites, and Final/Mid term mark distribution extraction.
- The response path includes language validation and an optional retry for wrong language responses.

15. Known Limitations

- Text-only PDF extraction works; scanned image PDFs require OCR.
- Fixed word-based chunking can split tables and long sections awkwardly.
- The fast extractor is strongest for synthetic or well-defined answer patterns and is less flexible across all possible question styles.
- Language detection is heuristic and can misclassify short mixed-language questions.
- Local GGUF generation requires model files, disk space, and a compatible CPU runtime.
- Retrieval scores are ranking signals rather than calibrated probabilities.
- Manual expert review is still needed for ground-truth factual and groundedness validation.

16. Recommended Future Improvements

1. Add unit and integration tests for answer-bank, fast extraction, retrieval reranking, and language-validation flows.
2. Add source-document and page-label annotations to the evaluation dataset.
3. Improve OCR and table-aware PDF parsing support.
4. Add a stronger answer schema for course metadata and structured result fields.
5. Add confidence and unsupported-answer diagnostics when retrieved contexts are weak.
6. Add a document-refresh/index-rebuild command that automatically detects changed PDFs.
7. Add automatic metrics using source/page annotation support.

17. End-to-End Summary

The project processes university PDFs into normalized, chunked text records; embeds those chunks; indexes them in FAISS; retrieves context from the vector store; and answers the question through a curated answer-bank check, structured extraction, fast extraction, or local GGUF generation. The pipeline validates the language of the output and returns the answer with source and page traceability for inspection and evaluation.